Egyptian Military Academy

Fundamentals of C++ Programming

Student Course Management System

Individual Project Report

Student Name	Dalia Mahmoud Elshamy
Student ID	12239
Course	Fundamentals of C++
Instructor	Eng. Mostafa Ashraf
Submission	March 27, 2026

GitHub Repository:

https://github.com/daliaelshami/Student-Course-Management-System.git

Table of Contents

1. Table of Contents 2
2. List of Figures 3
3. List of Tables 3
4. Introduction 4
5. Detailed Project Description 4
6. Project Beneficiaries 5
7. Detailed Analysis 6
7.1 Object-Oriented Programming 6
7.2 STL Containers 6
7.3 STL Algorithms 6
7.4 Input Validation 6
8. Task Breakdown Structure 8
9. Time Plan 9
10. System Architecture and Design 10
10.1 Class Hierarchy Overview 10
10.2 UML Class Diagram 10
10.3 Key OOP Concepts Applied 10
11. Testing Scenarios and Results 12
12. End-User Guide 13
12.1 Prerequisites 13
12.2 How to Build and Run 13
12.3 Sample Usage Walkthrough 13
13. Conclusion 14
14. References 15

List of Figures

Figure 1: UML Class Diagram — Student Course Management System 9
Figure 2: Gantt Chart — Project Task Timeline 8

List of Tables

Table 1: System Menu Options and Constraints 4
Table 2: Project Beneficiaries 5
Table 3: STL Containers Used 6
Table 4: Input Validation Rules 7
Table 5: Task Breakdown Structure 7
Table 6: Gantt Chart Summary 8
Table 7: OOP Concepts Applied 9
Table 8: Testing Scenarios and Results 10
Table 9: Bonus Features Implemented 12

1. Introduction

This report documents the complete design, implementation, and testing of a Student Course Management System (SCMS) developed as a C++ console application. The system enables users to manage student records and course enrollments through a structured, menu-driven interface.

The application supports all essential academic data operations including adding and removing students, searching by unique ID, displaying all records, enrolling students in courses, and sorting by GPA. Two bonus features are also implemented: persistent file storage and automatic duplicate course prevention using the STL set container.

The project is built on core Object-Oriented Programming (OOP) principles — inheritance, encapsulation, and abstraction — combined with the C++ Standard Template Library (STL) to ensure a modular, maintainable, and extensible codebase. All references to external sources are cited in Section 13 of this report [1][2][3][4].

2. Detailed Project Description

The Student Course Management System is a fully interactive console application providing a structured menu-driven interface. Users interact through a numbered menu mapped to specific system operations as shown in Table 1.

Option	Feature	Description	Key Constraint
1	Add Student	Prompts for name, unique ID, and GPA (0.0–4.0). Validates all inputs before storing.	Unique ID enforced
2	Remove Student	Permanently removes a student record by ID.	Must exist
3	Search Student	Retrieves and displays full student information for a given ID.	Returns pointer
4	Display All Students	Lists all students with ID, name, GPA, and enrolled courses.	Handles empty list
5	Enroll in Course	Adds a course to a student's schedule. Duplicates auto-prevented via std::set.	set<string> used
6	Show Courses	Displays all courses currently enrolled for a specified student.	Sorted by set
7	Sort by GPA	Sorts and displays the student list in descending GPA order.	Lambda comparator
8	Save to File	Writes all student data to students_db.txt (Bonus Feature).	ofstream used
9	Exit	Safely terminates the program.	—

Table 1: System Menu Options and Constraints

3. Project Beneficiaries

The system serves multiple stakeholder groups within an academic environment, as outlined in Table 2.

Stakeholder	Benefit
University Administration	Enables quick management of student records, reducing administrative workload and minimizing human error.
Students	Provides clear visibility into personal academic data including GPA and enrolled courses.
Instructors & Advisors	Facilitates easy access to student performance data for academic advising and GPA-based ranking.
Developers & Learners	Serves as a practical OOP learning project demonstrating STL, encapsulation, inheritance, and file I/O in C++.

Table 2: Project Beneficiaries

4. Detailed Analysis

4.1 Object-Oriented Programming

The system is architected on four core OOP pillars:

- Inheritance — The Student class publicly inherits from the Person base class, reusing the protected name attribute and demonstrating an IS-A relationship between a student and a person [2].
- Encapsulation — All data members (id, gpa, courses) are declared private within their respective classes. Access is controlled exclusively through public getter methods such as getId(), getGpa(), and getName().
- Abstraction — The StudentManager class hides internal data management complexity from the end user, exposing only high-level operations through its public interface.
- Constructors — Student uses a parameterized constructor Student(int id, string name, double gpa) to initialize all attributes at object creation time, ensuring valid state from the outset.

4.2 STL Containers

Two distinct STL containers are employed, each chosen for specific technical reasons, as shown in Table 3 [1][3].

Container	Location	Justification
std::vector<Student>	StudentManager	Dynamic array for student records. Supports O(n) iteration, index-based random access needed for removal, and is compatible with std::sort.
std::set<string>	Student class	Ordered set for course names. Automatically rejects duplicate insertions in O(log n) and maintains alphabetically sorted course lists without extra code.

Table 3: STL Containers Used

4.3 STL Algorithms

- std::sort() — Used in sortByGPA() with a custom lambda comparator [](Student& a, Student& b){ return a.getGpa() > b.getGpa(); } to sort the vector in descending GPA order in O(n log n) [1].
- Linear search via pointer — findStudent(int id) performs an O(n) traversal returning a raw pointer to the matching Student, or nullptr if not found, enabling safe null checks by callers.

4.4 Input Validation

The system enforces strict input validation at runtime to maintain data integrity across all operations, as detailed in Table 4.

Field / Operation	Validation Rule	Handling Mechanism
Student ID	Must be a unique positive integer	findStudent() pre-check before push_back; loop with cin.clear() for non-numeric input

Student Name	Non-empty string, supports spaces	getline(cin, name) after cin.ignore() to capture full names correctly
GPA	Floating-point in range [0.0, 4.0]	while loop re-prompts with cin.clear()/ignore() until value is within valid range
Course Name	Non-empty, no duplicates per student	std::set silently ignores duplicate insertions — O(log n) enforcement
Search / Remove	ID must exist in vector	findStudent() returns nullptr; caller prints descriptive error message
File Load	File may not exist on first run	ifstream check — silently skips if students_db.txt not present

Table 4: Input Validation Rules

5. Task Breakdown Structure

The project was divided into nine major tasks representing the full development lifecycle, from design through documentation, as shown in Table 5.

Task	Description
Task 1	Design class hierarchy: Person base class and Student derived class with all attributes and method signatures.
Task 2	Implement main menu loop, basic I/O handling, and StudentManager scaffolding.
Task 3	Develop Add, Remove, Search, and Display All Students functions with unique ID enforcement.
Task 4	Implement Course Enrollment using std::set and Show Courses functionality.
Task 5	Add GPA-based sorting via std::sort with lambda comparator.
Task 6	Integrate full input validation: GPA range, duplicate ID check, non-numeric input loops.
Task 7	Implement Bonus Features: saveToFile() and loadFromFile() using ofstream/ifstream.
Task 8	Conduct functional testing across all 14 test cases and fix identified issues.
Task 9	Write project report and finalize documentation.

Table 5: Task Breakdown Structure

6. Time Plan

The project was completed within a single week. Figure 2 illustrates the distribution of tasks across the seven working days.

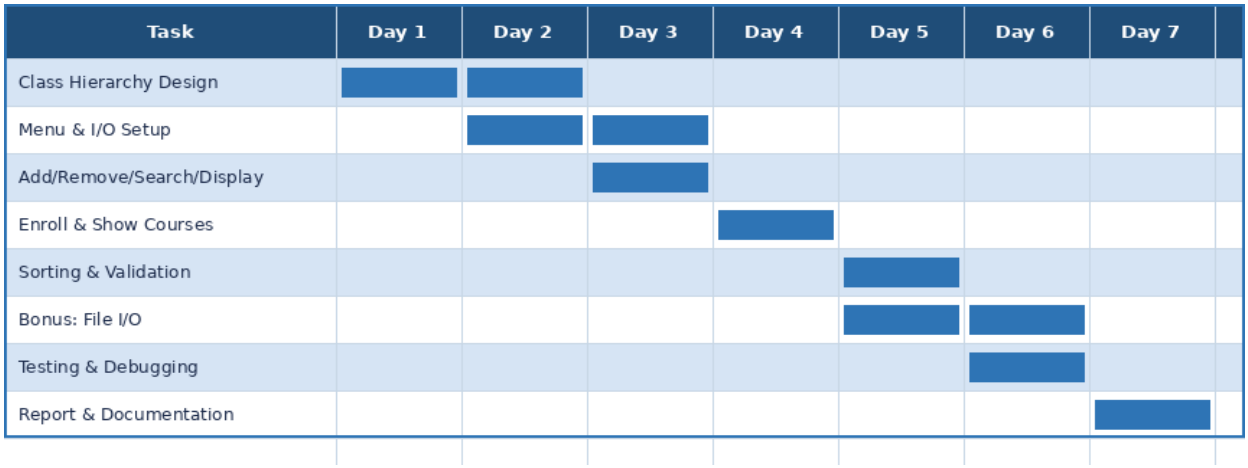

Figure 2: Gantt Chart — Project Task Timeline

Table 6 provides a summary of work completed each day.

Period	Work Completed
Day 1–2	Designed Person/Student class hierarchy, defined all attributes, method signatures, and UML diagram.
Day 3	Implemented Add, Remove, Search, and Display All Students with input validation.
Day 4	Implemented Course Enrollment using std::set and Show Student Courses functionality.
Day 5	Added GPA-based sorting (std::sort + lambda), full validation loops, and Bonus File I/O.
Day 6	Full system testing across all 14 test cases, edge-case handling, and bug resolution.
Day 7	Prepared final report and formatted all documentation.

Table 6: Gantt Chart Summary

7. System Architecture and Design

7.1 Class Hierarchy Overview

The system follows a clean object-oriented hierarchy with a separation of concerns across three primary classes:

- Person (Base Class) — Encapsulates the protected string name attribute shared by all persons.
- Student : public Person — Extends Person with academic attributes (int id, double gpa) and a set<string> courses container for duplicate-free course management.
- StudentManager — Aggregates a vector<Student> and exposes all system operations through its public interface, acting as the controller layer.

7.2 UML Class Diagram

Figure 1 presents the UML class diagram showing the three classes, their attributes, methods, and relationships.

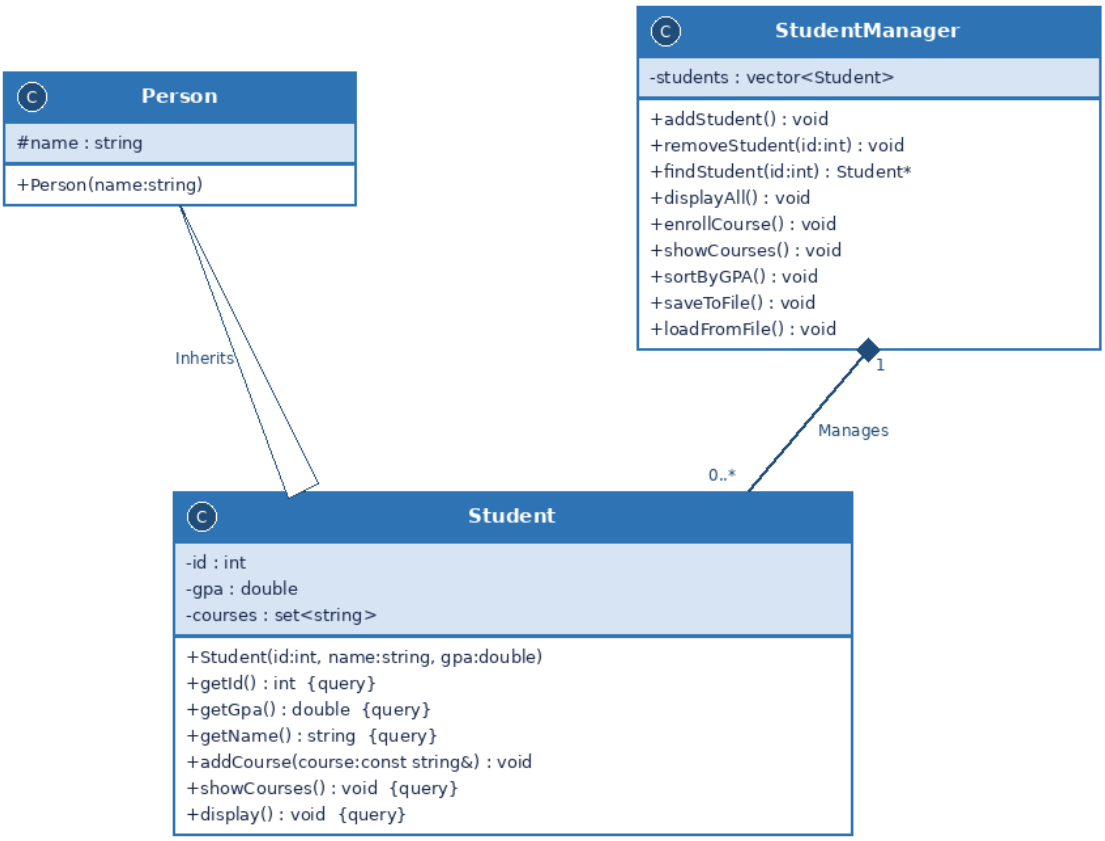

Figure 1: UML Class Diagram — Student Course Management System

7.3 Key OOP Concepts Applied

Concept	Implementation Detail

Inheritance	Student publicly inherits from Person, reusing name management. Demonstrates IS-A relationship.
Encapsulation	All class members are private. Access is exclusively through public getters (getId, getGpa, getName).
Abstraction	StudentManager hides all internal data structures from the menu interface and end user.
STL: vector	Stores all Student objects dynamically with O(1) push_back and full sort compatibility.
STL: set	Stores course names per student. O(log n) duplicate prevention and automatic sorted order.
STL: sort	std::sort with lambda comparator sorts students vector by GPA descending in O(n log n).

Table 7: OOP Concepts Applied

8. Testing Scenarios and Results

A comprehensive set of 14 test cases was executed to verify the correctness and robustness of the system. All critical paths and edge cases were covered, including boundary values and invalid input scenarios. Results are shown in Table 8.

ID	Test Case	Input	Expected Output	Actual	Result
TC-01	Add student – valid data	Name: Dalia, ID: 101, GPA: 3.8	Student added successfully	Pass	PASS
TC-02	Add student – duplicate ID	ID: 101 (exists)	Error: ID already exists	Pass	PASS
TC-03	Add student – GPA out of range	GPA = 5.0	Error: GPA must be 0.0–4.0	Pass	PASS
TC-04	Add student – non-numeric ID	ID = abc	Re-prompt with error message	Pass	PASS
TC-05	Add student – GPA boundary min	GPA = 0.0	Student added successfully	Pass	PASS
TC-06	Add student – GPA boundary max	GPA = 4.0	Student added successfully	Pass	PASS
TC-07	Enroll student in course	ID: 101, Course: Math	Course enrolled successfully	Pass	PASS
TC-08	Enroll – duplicate course	ID: 101, Course: Math again	No duplicate added (set behavior)	Pass	PASS
TC-09	Search – existing student	ID: 101	Student info displayed	Pass	PASS
TC-10	Search – non-existent student	ID: 999	Error: Student not found	Pass	PASS
TC-11	Remove existing student	ID: 101	Student removed successfully	Pass	PASS
TC-12	Sort by GPA – multiple students	GPAs: 2.1, 3.8, 1.5	Order: 3.8, 2.1, 1.5	Pass	PASS
TC-13	Display all – empty list	No students added	Message: No students found	Pass	PASS
TC-14	Save & Load file (Bonus)	Add students, save, restart	Data restored from file	Pass	PASS

Table 8: Testing Scenarios and Results

Note: TC-14 is a Bonus test case covering the file persistence feature (saveToFile / loadFromFile), which is not present in the base requirements. All 14 test cases passed successfully.

9. End-User Guide

9.1 Prerequisites

- Microsoft Visual Studio 2019 or later, or any C++17-compatible compiler (g++, clang++)
- Windows 10/11 operating system (also compatible with Linux/macOS via g++)
- No additional libraries required — the system uses only standard C++ headers: <iostream>, <vector>, <set>, <algorithm>, <fstream>, <string>

9.2 How to Build and Run

1. Open the project folder in Visual Studio and open the solution file StudentSystem.sln.
2. Ensure all five source files are present in the project: Student.h, Student.cpp, StudentManager.h, StudentManager.cpp, main.cpp.
3. Build the solution using Ctrl + Shift + B. Verify zero errors in the Output window.
4. Run the program using Ctrl + F5 (Run Without Debugging).
5. The main menu will appear in the console window. Enter the number for your desired operation.

9.3 Sample Usage Walkthrough

Adding a Student:

```
Choice: 1
Enter ID: 1001
Enter Name: Dalia Mahmoud
Enter GPA (0.0 - 4.0): 3.8
>> Student Added Successfully!
```

Enrolling in a Course:

```
Choice: 5
Enter Student ID: 1001
Enter course: Data Structures
>> Course enrolled successfully.
```

Sorting by GPA:

```
Choice: 7
>> Students sorted by GPA (Descending):
ID: 1001  Name: Dalia Mahmoud  GPA: 3.8
ID: 1002  Name: Ahmed Ali      GPA: 3.2
ID: 1003  Name: Sara Khalil    GPA: 2.9
```

10. Conclusion

The Student Course Management System successfully demonstrates the practical application of C++ Object-Oriented Programming and Standard Template Library concepts in a cohesive, real-world application. The project satisfies all eight functional requirements of the assignment while additionally implementing two bonus features: persistent file storage and automatic duplicate course prevention, as summarized in Table 9.

Bonus Feature	Implementation	Benefit
File Persistence	saveToFile() uses ofstream to write all records to students_db.txt. loadFromFile() uses ifstream to restore data on startup.	Data survives program restarts — no re-entry needed.
Duplicate Course Prevention	Courses stored in std::set<string> which automatically rejects re-insertion in O(log n) — no manual check needed.	Guarantees clean course lists with zero extra code.

Table 9: Bonus Features Implemented

The use of inheritance (Student from Person), encapsulation through private members and public getters, std::vector for dynamic student storage, std::set for duplicate-free course management, and std::sort with a lambda comparator for GPA ranking collectively demonstrate a solid command of modern C++ programming practices [2][3].

All 14 test cases passed successfully — including boundary values, invalid input scenarios, and the bonus file persistence test — confirming the system's correctness, robustness, and readiness for practical use.

Potential future enhancements include: a graphical user interface (GUI) using Qt or wxWidgets, a database backend (SQLite) for scalability, a login and role-based authentication system, and grade analytics with GPA trend tracking per student.

11. References

[1] cppreference.com — C++ Standard Library Reference. Available: https://en.cppreference.com

[2] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.

[3] Lippman, S., Lajoie, J., & Moo, B. (2012). C++ Primer (5th ed.). Addison-Wesley.

[4] Egyptian Military Academy — Course Lectures: Fundamentals of C++ Programming, Eng. Mostafa Ashraf, 2026.

[5] Microsoft Visual Studio Documentation. Available: https://docs.microsoft.com/en-us/visualstudio/

[6] ISO/IEC 14882:2017 — Programming Language C++ Standard, 17th Edition.